

基于拓扑-几何合并树的节点简化研究

核心目标是：

在保留合并树关键拓扑结构的前提下，从增广合并树的大量普通节点中筛选出真正代表几何或结构变化的节点，从而减少冗余，并保留主要等值面演化过程。

1. 问题背景

合并树用于描述标量场中连通分量随标量值变化而产生、合并和消亡的过程。临界节点对应重要拓扑事件，树上的弧或拓扑边表示两个拓扑事件之间的连续演化。

实际数据中，为了更细致地记录等值面变化，合并树常被增广，在临界节点之间插入大量普通节点。这些普通节点可以记录等值面沿标量方向的变化，但也会带来冗余：

- 树结构过密，主要拓扑分支不够突出。
- 相邻普通节点对应的等值面可能非常相似。
- 可视化时会出现大量重复或近似的等值面。
- 后续分析、存储和交互成本增加。

因此，本研究关注的不是删除临界节点，而是在保留临界节点和主要拓扑结构的前提下，对拓扑链内部的普通节点进行几何或结构去冗余。

2. 研究动机

几何角度

从几何角度看，增广合并树中的普通节点可以理解为等值面演化过程中的采样点。沿着同一条拓扑链，标量阈值逐渐变化，对应的局部等值面或局部等值区域也连续变化。

但并不是每个普通节点都对应明显不同的几何状态。很多相邻节点只表现为等值面的轻微移动、平滑膨胀或局部小扰动。这些节点在几何上高度相似，如果全部保留下来，就会造成冗余。

另一方面，只保留临界节点又过于粗糙。两个拓扑事件之间虽然没有新的连通性变化，但等值面形状仍可能发生明显变化，例如：

- 局部体积明显增大或缩小。
- 形状从紧凑变成长条。
- 局部出现细颈、延展或空间偏移。
- 等值面穿过了新的原始网格区域。

这些变化未必产生新的拓扑事件，但对理解标量场结构仍然重要。因此，我们希望在每条拓扑链内部选出少量“几何代表节点”，用较少节点表达主要形态变化。

拓扑角度

从拓扑角度看，合并树的核心语义来自临界节点和分支结构。极值点、鞍点、根节点、叶节点和分支节点不能被随意删除，否则会破坏合并树对标量场连通性演化的表达。

因此，简化不能只根据几何相似性在全局范围内做。不同拓扑分支上可能存在形状相似的等值面，但它们在合并树中的语义不同，不能互相替代。

拓扑结构在这里提供三个约束：

1. 临界节点和结构节点必须保留。
2. 普通节点的筛选应限制在同一条拓扑链或同一局部拓扑区域内。
3. 删除普通节点后，需要重建简化树，使结果仍是一棵合法的合并树。

可以概括为：

拓扑结构决定哪些节点不能删
拓扑链决定哪些节点可以比较
几何或结构度量决定链内普通节点是否保留

3. 比较对象：不是全局等值面

比较两个节点时，不应该提取同一标量值下的全局所有等值面，也不是把所有等值面取平均后比较。

原因是，同一个标量值 τ 下，标量场中可能存在多个不连通等值面，其中一些属于其他拓扑分支或其他拓扑链。如果全部拿来比较，就会把无关结构混入当前节点。

更合理的比较对象是：

当前节点在所属拓扑链、局部分割区域或局部拓扑分支内诱导出的局部等值结构。

•

方法	筛选是否在当前 BRANCH 内	描述符/局部区域怎么限定	是否严格只针对当前分支
IoU 当前代码	是	buildBranchLocalVolume() 只遍历 branch.nodeIds	是
ISM TXT	是	node.subtreeVertices 的局部 mask 和距离场	否，可能包含旁支
ActiveGraph VTU/C++	是	当前 chain 相邻 arc 的 SegmentationId，再过滤 active edges	基本是
ActiveGraph TXT	是	node.subtreeVertices 内的 active edges	否，可能包含旁支
GNN Python	是	VTU 用 node incident segIds；TXT 用 local_region/subtree	

4. 总体简化框架

三种策略都可以放入同一个链上贪心框架。

对每条拓扑链：

```
v_0, v_1, ..., v_k
```

其中 v_0 和 v_k 是结构节点，必须保留。中间节点是候选普通节点。

简化流程：

```

keep = [v_0]
ref = v_0

for v_i in v_1 ... v_{k-1}:
    计算 ref 和 v_i 的局部等值结构差异 Dist(ref, v_i)
    如果 Dist 超过阈值:
        保留 v_i
        ref = v_i

keep.append(v_k)

```

因此，关键问题变成：

如何定义两个链内节点之间的局部几何或结构差异？

当前主要考虑三种策略。

5. 三种基础策略

策略一：基于 ISM 的局部形状相似性

ISM 方法关注局部等值结构的形状差异。

基本思路是：不直接比较等值面三角网格，而是把局部等值结构转换成距离场或类似的空间描述，再通过互信息衡量两个节点的相似性。

对节点 v ：

```

局部区域  $\Omega(v)$ 
    ↓
阈值  $f(v)$ 
    ↓
局部等值边界或等值面
    ↓
距离场描述符

```

两个节点之间可定义：

$$\text{Dist_ISM} = 1 - \text{NMI}(\text{D_ref}, \text{D_candidate})$$

其中 **NMI** 是距离场之间的归一化互信息。

ISM 的优点是能捕捉体积相近但形状不同的情况，适合强调几何保真。缺点是计算较重，参数较多；如果局部区域内有多个不连通分量，整体距离场可能会把它们混在一起，因此需要后续的多分支处理。

策略二：基于局部体积 IoU 的快速比较

IoU 方法关注局部子水平集体积变化。

对节点 **v**，在局部区域 **Omega(v)** 内计算对应体积：

$$v(v) = \text{volume}(\{x \text{ in } \Omega(v) \mid f(x) \leq f(v)\})$$

两个节点之间的体积相似性可以写成：

$$\text{IoU} = \min(v_{\text{ref}}, v_{\text{candidate}}) / \max(v_{\text{ref}}, v_{\text{candidate}})$$

差异为：

$$\text{Dist_vol} = 1 - \text{IoU}$$

当：

$$\text{Dist_vol} \geq \text{epsilon_vol}$$

则保留候选节点。

IoU 的优点是实现简单、参数直观，适合作为 **branch** 内的快速初筛。缺点是只看体积，不直接看形状；对于“体积相近但形态变化明显”的情况不敏感。

策略三：基于活性子图的结构比较与 GNN 向量化

活性子图方法关注等值面在原始网格中截穿了哪些边。

对于原始网格边 (u, w) ，如果：

```
f(u) <= t < f(w)
or
f(w) <= t < f(u)
```

则这条边是阈值 t 下的活性边。

对节点 v ：

```
t = f(v)
```

只在局部区域 $\Omega(v)$ 中提取活性边，得到活性子图：

```
G(v) = (V_active(v), E_active(v))
```

最直接的比较方式是活性边集合的 Jaccard 距离：

```
Dist_J = 1 - |E_ref intersect E_candidate| / |E_ref union E_candidate|
```

也可以进一步使用 GNN 把局部活性子图编码成固定长度向量：

```
局部活性子图
  ↓
节点特征
  ↓
GCN / GNN 编码
  ↓
图级 embedding
  ↓
cosine distance
```

GNN 版和 ActiveGraph 版共享局部活性子图定义，只是前者把显式边集合编码成向量，再用 embedding 的 cosine distance 做比较。

这一类方法的优点是更贴近原始离散网格结构，不依赖完整等值面重建；GNN 版本还能把局部结构、空间位置、标量和分割信息编码进统一向量。缺点是解释性弱于显式边集合，如果使用未训练的随机权重，GNN 更适合作为结构化 baseline，而不是最终学习模型。

6. 三种策略之后：按拓扑树处理多分支

单分支、双分支、三分支、多分支的处理可以放在三种基础策略之后，作为一个统一模块。

这里不需要再做分支匹配。原因是分类和分支身份已经由拓扑树给出：当前节点位于哪条链、附近连接了哪些子分支、哪些是主分支或旁支，都可以从合并树结构中得到。

因此，多分支处理的流程应写成：

```
先用 ISM / IoU / ActiveGraph / GNN 定义单个局部结构的差异
↓
再根据拓扑树判断局部结构属于单分支、双分支、三分支还是多分支
↓
对每一类选取一个或若干代表分支
↓
计算各代表分支上的局部差异
↓
用权重汇总为当前节点的整体差异
```

可以记为：

$$\mathbf{B}(v) = \{b_1(v), b_2(v), \dots, b_k(v)\}$$

其中 $\mathbf{B}(v)$ 是由拓扑树确定的当前节点附近的局部分支集合，不是任意连通分量集合。

单分支

当前局部结构只有一个主要分支，直接比较该分支上的代表局部结构：

$$D(v_{\text{ref}}, v) = d_1(v_{\text{ref}}, v)$$

双分支

当前局部结构由拓扑树判断为两个分支，例如一个主分支和一个旁支。此时沿拓扑树给出的分支身份分别比较：

$$D(v_{\text{ref}}, v) = w_1 d_1 + w_2 d_2$$

其中 d_1 、 d_2 是两个分支上的局部差异， w_1 、 w_2 是两个分支的贡献权重。

三支

三支情况可以理解为一个局部结构附近有三个代表方向或三个子分支。处理方式仍然是按拓扑树给出的分支身份分别计算：

$$D(v_{\text{ref}}, v) = w_1 d_1 + w_2 d_2 + w_3 d_3$$

这类情况适合观察多个旁支是否同时发生变化。如果某个分支几何规模不大但拓扑重要性高，它的权重不应被简单平均或几何规模压低。

多分支

当局部分支数超过三个时，不建议把所有分支完全平均。可以先按拓扑树和权重指标筛出代表分支：

多分支集合
↓
按拓扑重要性、几何规模、局部体积或 **GNN** 重要性排序
↓
保留前 **K** 个代表分支
↓
其余弱分支合并为 **residual** 项或低权重项

整体差异可以写成：

$$D(v_{\text{ref}}, v) = \sum_i w_i d_i + w_{\text{res}} d_{\text{res}}$$

其中 d_{res} 表示剩余小分支的整体变化。如果不希望引入 `residual`，也可以只保留权重大于阈值的分支：

```
w_i >= tau_branch
```

这样可以避免大量小噪声分支干扰最终判断。

7. 多分支权重怎么考虑

多分支比较的关键是权重。“拓扑树分类后，每个代表分支对整体差异的贡献权重”。

基本形式为：

```
D(v_ref, v) = sum_i w_i d_i
```

其中：

- d_i 是第 i 个代表分支上的局部差异。
- w_i 是该分支的重要性权重。
- $\sum_i w_i = 1$ 。

不能简单平均，因为小噪声分支不应和主分支同等重要；但也不能只按大小加权，否则小而重要的拓扑分支会被忽略。

几何规模权重

几何规模权重关注分支对应的局部结构有多大。可以使用：

- 等值面面积。
- 活性边数 $|E_i|$ 。
- 局部点数或体素数。
- 包围盒尺度。

例如对活性子图：

```
w_i = |E_i| / sum_j |E_j|
```

为了避免最大分支过度主导，可以使用压缩后的规模：

$$\text{size}_i = \sqrt{|E_i|}$$

或：

$$\text{size}_i = \log(1 + |E_i|)$$

局部体积权重

如果分支对应局部区域，可以使用局部体积：

$$w_i = \text{volume}_i / \sum_j \text{volume}_j$$

这适合与 IoU 方法和 `RegionSize` 结合。它比单纯面积更稳定，因为它反映的是该分支控制的局部空间规模。

拓扑重要性权重

如果分支对应不同拓扑分支，可以使用拓扑重要性：

$$w_i = \text{persistence}_i / \sum_j \text{persistence}_j$$

也可以使用：

$$w_i = \text{RegionSpan}_i / \sum_j \text{RegionSpan}_j$$

这样可以避免面积不大但拓扑上重要的分支被忽略。

GNN 重要性权重

如果使用 GNN 活性子图向量化方法，权重还可以由 GNN embedding 给出。

一种做法是使用结构显著性。设第 i 个分支的 GNN 向量为 z_i ，可以用它相对于局部平均结构的偏离程度作为重要性：

```
gnn_i = ||z_i - mean_j(z_j)||  
w_i = gnn_i / sum_j gnn_j
```

如果一个分支的活性子图结构明显不同，它的 **embedding** 会更偏离局部平均，因此权重更高。

另一种做法是使用变化显著性。比较参考节点和候选节点在同一拓扑分支上的 **embedding**：

```
gnn_i = 1 - cosine(z_i(ref), z_i(candidate))  
w_i = gnn_i / sum_j gnn_j
```

这种权重会更强调“当前真正发生明显结构变化的分支”。如果某个分支本身很大但变化很小，它不会因为规模大而自动占据过高权重。

如果后续 GNN 模型进一步训练，也可以让模型直接输出分支 **importance score**：

```
a_i = MLP(z_i)  
w_i = softmax(a_i)
```

这个版本更像学习型权重，需要训练数据或伪标签支撑。当前阶段可以先使用前两种无监督 GNN 权重。

混合权重

实际更推荐使用混合权重：

```
raw_w_i = sqrt(size_i) * topo_i * (1 + gnn_i)  
w_i = raw_w_i / sum_j raw_w_j
```

其中：

- **size_i** 表示几何规模，可以来自面积、活性边数、点数或体素数。
- **topo_i** 表示拓扑重要性，可以来自 **persistence**、**RegionSpan** 或分支跨度。
- **gnn_i** 表示 GNN 结构显著性或变化显著性。

这里使用 `sqrt(size_i)` 是为了避免大分支过度主导；使用 `(1 + gnn_i)` 是为了让 GNN 只增强重要分支，而不是把某些分支完全抹掉。

如果暂时不用 GNN 权重，可以退化为：

```
raw_w_i = sqrt(size_i) * topo_i  
w_i = raw_w_i / sum_j raw_w_j
```

8. 最终方法架构

最终方法可以理解为两层。

第一层是基础节点差异策略：

ISM：比较局部形状
IoU：比较局部体积
ActiveGraph/GNN：比较原始网格活性子图结构

第二层是多分支统一处理：

根据拓扑树判断局部分支类型
↓
单分支直接比较
↓
双分支、三分支分别计算各代表分支差异并加权汇总
↓
多分支先选择代表分支，再加入 `residual` 或低权重项
↓
权重由几何规模、局部体积、拓扑重要性和 GNN 显著性共同决定

9. 具体怎么衔接和实现

上面提出的三类方法不是三套互不相关的流程，而是放在同一个简化框架中的三种“局部分支差异计算器”。它们的共同输入都是：

参考节点 `v_ref`
候选节点 `v`
当前拓扑链或局部拓扑区域 `Omega`
由拓扑树得到的局部分支类型 `B(v)`

它们的共同输出都是：

当前候选节点相对于参考节点的差异 $D(v_{\text{ref}}, v)$

也就是说，三类方法的区别只在于“每个分支上的差异 d_i 怎么算”，而不是改变整个简化流程。

9.1 第一步：先由拓扑树确定可比较范围

首先从合并树中提取拓扑链。每条链两端是临界节点或结构节点，中间是普通节点：

$v_0, v_1, \dots, v_k$

其中 v_0 和 v_k 必须保留。中间节点是候选节点。

对每个候选节点 v_i ，先根据拓扑树确定它的局部区域：

$\Omega(v_i)$

这个区域可以来自：

- 当前拓扑链对应的局部区域。
- 当前节点 incident arc 的 `SegmentationId`。
- 当前节点的 subtree/local region。
- 当前分支附近的局部分割区域。

这一步的作用是限制比较范围，避免把同一标量值下其他拓扑分支上的等值面混进来。

9.2 第二步：由拓扑树判断分支类型

在局部区域确定之后，再根据拓扑树判断当前局部结构属于哪一类：

单分支 / 双分支 / 三分支 / 多分支

这里的分类不是靠几何匹配做出来的，而是直接来自拓扑树的连接关系。例如：

- 当前局部区域只有一条主要拓扑链，就是单分支。
- 当前节点附近有一个主分支和一个旁支，就是双分支。
- 当前节点附近有三个子分支或三个代表方向，就是三分支。
- 超过三个局部分支，就是多分支。

得到分支集合：

$$B(v_i) = \{b_1, b_2, \dots, b_m\}$$

后续每个分支分别计算差异，再加权汇总。

9.3 第三步：在每个分支上计算差异

对第 i 个分支 b_i ，从参考节点和候选节点分别提取该分支上的局部结构：

$$\begin{aligned} &S_i(v_{ref}) \\ &S_i(v) \end{aligned}$$

然后使用三类方法之一计算分支差异。

如果使用 ISM：

$$\begin{aligned} &S_i(v_{ref}), S_i(v) \\ &\quad \downarrow \\ &\text{局部距离场 } D_i(ref), D_i(v) \\ &\quad \downarrow \\ &d_i = 1 - NMI(D_i(ref), D_i(v)) \end{aligned}$$

如果使用局部体积 IoU：

$$\begin{aligned} &S_i(v_{ref}), S_i(v) \\ &\quad \downarrow \\ &\text{局部体积 } v_i(ref), v_i(v) \\ &\quad \downarrow \\ &d_i = 1 - \min(v_i(ref), v_i(v)) / \max(v_i(ref), v_i(v)) \end{aligned}$$

如果使用 ActiveGraph/GNN:

```
S_i(v_ref), S_i(v)
  ↓
局部活性子图 G_i(ref), G_i(v)
  ↓
ActiveGraph: d_i = 1 - Jaccard(E_i(ref), E_i(v))
GNN:         d_i = 1 - cosine(z_i(ref), z_i(v))
```

因此三类方法在这里衔接到同一个接口:

```
分支 b_i
  ↓
计算分支差异 d_i
```

9.4 第四步：根据分支类型汇总

如果是单分支:

```
D(v_ref, v) = d_1
```

如果是双分支:

```
D(v_ref, v) = w_1 d_1 + w_2 d_2
```

如果是三分支:

```
D(v_ref, v) = w_1 d_1 + w_2 d_2 + w_3 d_3
```

如果是多分支:

```
D(v_ref, v) = sum_i w_i d_i + w_res d_res
```

其中 w_i 来自几何规模、局部体积、拓扑重要性和 GNN 显著性; d_{res} 表示没有单独展开的小分支整体变化。

这一步是三类方法之后的统一处理层。无论前面用 ISM、IoU 还是 ActiveGraph/GNN，最后都进入同一个加权汇总公式。

9.5 第五步：用汇总差异决定是否保留节点

得到整体差异 $D(v_{ref}, v_i)$ 后，进入链上贪心筛选：

```
keep = [v_0]
ref = v_0

for v_i in v_1 ... v_{k-1}:
    确定  $\Omega(v_i)$ 
    根据拓扑树判断分支类型
    对每个代表分支计算  $d_i$ 
    根据权重汇总得到  $D(ref, v_i)$ 

    if  $D(ref, v_i) \geq \epsilon$ :
        keep.append(v_i)
        ref = v_i

keep.append(v_k)
```

这样，三类方法和多分支处理的关系就变成：

```
拓扑树给出比较范围和分支类型
↓
ISM / IoU / ActiveGraph-GNN 负责计算每个分支上的差异
↓
多分支权重负责把多个  $d_i$  汇总为  $D$ 
↓
链上贪心根据  $D$  决定普通节点是否保留
```

9.6 三类方法之间怎么使用

实验和实现上可以分三种层次。

第一层是并列比较。分别运行三套版本：

ISM 简化结果
IoU 简化结果
ActiveGraph/GNN 简化结果

这样可以观察三种度量各自保留了哪些节点，以及它们对形状、体积和原始网格结构变化的敏感性。

第二层是级联使用。可以把 IoU 作为快速初筛，把 ActiveGraph/GNN 作为结构变化判断，把 ISM 作为高精度确认：

IoU 判断局部体积是否明显变化
↓
若体积变化不明显，再看 ActiveGraph/GNN 是否发生结构变化
↓
若仍不确定，再用 ISM 判断形状是否明显变化

这种方式比较节省计算，因为 ISM 最重，不需要对所有节点都计算。

第三层是融合使用。把三种差异归一化后组合成一个分支差异：

$$d_i = \lambda_1 d_i^{\text{ISM}} + \lambda_2 d_i^{\text{IoU}} + \lambda_3 d_i^{\text{Graph}}$$

其中：

$$\lambda_1 + \lambda_2 + \lambda_3 = 1$$

然后再进入多分支加权：

$$D(v_{\text{ref}}, v) = \sum_i w_i d_i$$

当前阶段建议先做第一层和第二层：先让三种方法分别跑通并比较结果，再考虑是否需要融合成一个最终指标。这样实验更清楚，也更容易说明每种方法的作用。

10. 当前实验结果简要概览

Tangaroa 数据

原始节点数为 593，原始弧数为 592。

ISM 阈值实验：

参数组	保留节点
very_strict	517 / 593
strict	465 / 593
loose	421 / 593
very_loose	356 / 593

IoU 参数实验：

这里的数值来自旧版 IoU 原型结果；由于当前代码已经改成局部体积 IoU，若要给出最终结论，需要按新代码重新跑一遍实验。

EPS	保留节点
0.15	394 / 593
0.35	348 / 593
0.55	316 / 593
0.75	287 / 593

ActiveGraph 参数实验：

参数组	保留节点
strict	436 / 593
loose	260 / 593

这些结果说明，三种策略都可以通过阈值控制简化强度，但保留节点的原因不同：

- ISM 更关注形状变化。
- IoU 更关注局部体积变化。
- ActiveGraph/GNN 更关注原始网格活性子图结构变化。

TXT case17 数据

原始节点数为 420。

方法	保留节点
ISM	128 / 420
ActiveGraph	153 / 420
IoU, eps=0.45	71 / 420

TXT case20 数据

原始节点数为 500。

方法	保留节点
ISM	118 / 500
ISM sparse	43 / 500

11. 总结

本研究的核心是：在拓扑约束下做几何感知的合并树普通节点简化。

拓扑层面，临界节点和分支结构必须保留，普通节点筛选应限制在同一拓扑链或同一局部拓扑区域内。几何层面，链内普通节点只有在局部等值结构发生明显规模、形状、位置或原始网格截交结构变化时才需要保留。

三种基础策略分别从不同角度定义节点差异：

- ISM：局部形状相似性。
- IoU：局部体积变化。
- ActiveGraph/GNN：原始网格活性子图结构变化。

在三种策略之后，可以统一处理单分支、双分支、三分支和多分支情况。这里不需要重新做分支匹配，而是直接利用拓扑树给出的分支分类和分支身份。单分支直接比较；双分支和三分支分别计算各代表分支差异后加权汇总；多分支先选择代表分支，再对其余弱分支做 residual 或低权重处理。权重可以综合几何规模、局部体积、拓扑重要性和 GNN 显著性。这样既避免全局等值面混入，也避免多个局部分支被简单平均后丢失重要变化。